

Functional Interface and Lambda Expression in Java

1. What is a Functional Interface?

A **Functional Interface** is an interface that contains **exactly one abstract method**.

For example:

```
@FunctionalInterface
interface Operations {
    double operate(int a, int b);
}
```

Here:

- `Operations` is an interface.
- `operate()` is the only abstract method.
- It accepts two `int` values.
- It returns a `double`.
- Therefore, `Operations` is a **Functional Interface**.

The annotation:

```
@FunctionalInterface
```

tells the compiler that the interface is intended to contain exactly one abstract method.

If another abstract method is added, the compiler will report an error.

2. Why Do We Need Functional Interfaces?

Before Lambda Expressions, if we wanted to provide an implementation of an interface, we commonly used an **anonymous class**.

For example:

```
Operations obj = new Operations() {

    @Override
    public double operate(int a, int b) {
        return Math.pow(a, b);
    }
}
```

```
};
```

This works, but it requires a lot of code for implementing only one method.

Lambda expressions provide a shorter way to implement a Functional Interface.

The same implementation can be written as:

```
Operations obj = (int a, int b) -> Math.pow(a, b);
```

Thus:

Lambda Expression = A concise way of providing the implementation of a Functional Interface.

3. Basic Syntax of Lambda Expression

The general syntax is:

```
(parameters) -> expression
```

or

```
(parameters) -> {  
    statements;  
}
```

For example:

```
(a, b) -> a + b
```

Here:

- $(a, b) \rightarrow$ parameters
 - $-> \rightarrow$ lambda operator
 - $a + b \rightarrow$ expression/body
-

4. Complete Example

```
package com.company.lambda;
```

```
@FunctionalInterface
```

```

interface Operations {

    double operate(int a, int b);
}

public class Demo1 {

    public static void main(String[] args) {

        // 1. Anonymous Class
        Operations obj1 = new Operations() {

            @Override
            public double operate(int a, int b) {
                return Math.pow(a, b);
            }
        };

        System.out.println("Power = " + obj1.operate(2, 10));

        // 2. Lambda Expression with block body
        Operations obj2 = (int a, int b) -> {

            return a + b;
        };

        System.out.println("Addition = " + obj2.operate(10, 200));

        // 3. Lambda Expression with expression body
        Operations obj3 = (int a, int b) -> Math.pow(a, b);

        System.out.println("Power = " + obj3.operate(2, 10));

        // 4. Lambda Expression with inferred parameter types
        Operations obj4 = (a, b) -> a * b;

        System.out.println("Multiplication = " + obj4.operate(10, 20));
    }
}

```

Output

```

Power = 1024.0
Addition = 210.0
Power = 1024.0
Multiplication = 200.0

```

5. Understanding the Code Step by Step

Step 1 – Define the Functional Interface

```
@FunctionalInterface
interface Operations {

    double operate(int a, int b);
}
```

The interface defines the **contract**.

It says:

Any implementation of `Operations` must provide a method named `operate()` that accepts two integers and returns a double.

Step 2 – Implement Using Anonymous Class

```
Operations obj1 = new Operations() {

    @Override
    public double operate(int a, int b) {
        return Math.pow(a, b);
    }
};
```

Here we create an **anonymous class** that implements `Operations`.

The object `obj1` refers to that implementation.

Then:

```
obj1.operate(2, 10);
```

calls the implemented method.

Result:

```
1024.0
```

because:

```
2^10 = 1024
```

6. Replacing Anonymous Class with Lambda

The above anonymous class contains a lot of boilerplate code.

Because `Operations` is a Functional Interface, we can replace it with a Lambda Expression.

Instead of:

```
Operations obj1 = new Operations() {  
    @Override  
    public double operate(int a, int b) {  
        return Math.pow(a, b);  
    }  
};
```

we can write:

```
Operations obj1 = (int a, int b) -> Math.pow(a, b);
```

This is much shorter.

The compiler understands that the lambda is implementing:

```
double operate(int a, int b);
```

7. Lambda Expression with Block Body

A lambda can contain multiple statements.

For example:

```
Operations obj2 = (int a, int b) -> {  
    double result = a + b;  
    return result;  
};
```

Here `{ }` represents the **lambda body**.

Because the method returns `double`, the lambda must return a value.

8. Lambda Expression with Expression Body

If the lambda contains only one expression, braces and `return` can be omitted.

Instead of:

```
Operations obj2 = (int a, int b) -> {  
    return a + b;  
};
```

we can write:

```
Operations obj2 = (int a, int b) -> a + b;
```

Both are equivalent.

9. Type Inference in Lambda Expressions

Java can infer the parameter types from the Functional Interface.

Therefore:

```
Operations obj4 = (int a, int b) -> a * b;
```

can be simplified to:

```
Operations obj4 = (a, b) -> a * b;
```

The compiler already knows that `a` and `b` are `int` because the interface contains:

```
double operate(int a, int b);
```

Important Rule

We generally use either:

```
(int a, int b) -> a + b
```

or:

```
(a, b) -> a + b
```

Do not mix the two forms:

```
(int a, b) -> a + b    // Invalid
```

10. What Does the -> Operator Mean?

The -> is called the **lambda operator**.

It separates:

Parameters → Body

For example:

```
(a, b) -> a + b
```

can be read as:

Take a and b as input and return a + b.

Another example:

```
(a, b) -> Math.pow(a, b)
```

means:

Take a and b and return a raised to the power of b.

11. Important Concept: Lambda Does Not Work with Every Interface

A lambda expression can be assigned only where Java expects a **Functional Interface**.

For example:

```
Operations obj = (a, b) -> a + b;
```

works because `Operations` has one abstract method.

But:

```
interface Test {  
    void method1();  
    void method2();  
}
```

is **not** a Functional Interface because it contains two abstract methods.

Therefore:

```
Test obj = () -> System.out.println("Hello");
```

is invalid.

12. `@FunctionalInterface` Annotation

Consider:

```
@FunctionalInterface
interface Operations {

    double operate(int a, int b);
}
```

The annotation is not what makes an interface functional.

The interface is functional because it has **exactly one abstract method**.

The annotation tells the compiler:

"I intend this interface to be a Functional Interface. Please verify it."

For example:

```
@FunctionalInterface
interface Operations {

    double operate(int a, int b);

    double calculate(int a, int b);
}
```

This will produce a compilation error because there are two abstract methods.

13. Functional Interface Can Have Default and Static Methods

A Functional Interface can have:

- Exactly one abstract method
- Any number of default methods
- Any number of static methods

For example:

```
@FunctionalInterface
interface Operations {

    double operate(int a, int b);

    default void message() {
        System.out.println("Performing operation...");
    }

    static void information() {
        System.out.println("Operations Interface");
    }
}
```

It is still a Functional Interface because it has only **one abstract method**.

14. Real-World Analogy

Suppose a company defines:

```
interface Calculator {
    int calculate(int a, int b);
}
```

The interface does not specify **how** the calculation should be performed.

Different implementations can perform different operations:

```
Calculator add = (a, b) -> a + b;
```

```
Calculator multiply = (a, b) -> a * b;
```

```
Calculator subtract = (a, b) -> a - b;
```

```
Calculator maximum = (a, b) -> Math.max(a, b);
```

Now:

```
System.out.println(add.calculate(10, 20));
System.out.println(multiply.calculate(10, 20));
System.out.println(subtract.calculate(10, 20));
System.out.println(maximum.calculate(10, 20));
```

Output:

```
30
200
-10
20
```

This is an excellent example for explaining how the **same Functional Interface can have different implementations**.

15. Anonymous Class vs Lambda Expression

Anonymous Class

```
Operations obj = new Operations() {

    @Override
    public double operate(int a, int b) {
        return a + b;
    }
};
```

Lambda Expression

```
Operations obj = (a, b) -> a + b;
```

Anonymous Class	Lambda Expression
More code	Less code
Explicit implementation	Concise implementation
Creates an anonymous class	Represents behavior for a Functional Interface
Uses <code>new</code>	Does not use <code>new</code>
Requires method declaration	Method declaration is inferred from the interface

16. A Better Classroom Example

For teaching students, I recommend first using a simple `Calculator` example rather than `Math.pow()`.

```
@FunctionalInterface
interface Calculator {

    int calculate(int a, int b);
}
```

```

public class LambdaDemo {

    public static void main(String[] args) {

        Calculator add = (a, b) -> a + b;

        Calculator subtract = (a, b) -> a - b;

        Calculator multiply = (a, b) -> a * b;

        Calculator divide = (a, b) -> a / b;

        System.out.println("Addition = " + add.calculate(20, 10));

        System.out.println("Subtraction = " + subtract.calculate(20, 10));

        System.out.println("Multiplication = " + multiply.calculate(20, 10));

        System.out.println("Division = " + divide.calculate(20, 10));

    }
}

```

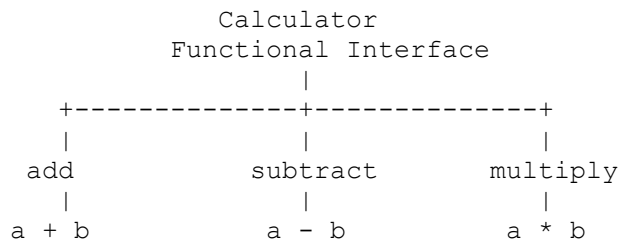
Output

```

Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2

```

This example makes the relationship particularly clear:



17. Key Points for Students

Students should remember these points:

1. A **Functional Interface** has exactly one abstract method.
2. `@FunctionalInterface` is used to let the compiler verify the intended design.
3. A **Lambda Expression** provides an **implementation of a Functional Interface**.
4. Lambda syntax uses the `->` operator.
5. Lambda parameters can generally use inferred types.
6. A single-expression lambda does not require `{}` or `return`.

7. A block-body lambda requires `return` when the functional method returns a value.
 8. Lambda expressions reduce boilerplate code.
 9. Lambda expressions are widely used with the **Java Collections and Stream API**.
 10. A lambda expression cannot be directly assigned to an ordinary class; it needs a compatible **Functional Interface**.
-

18. One-Line Concept for the Classroom

A useful way to explain the complete concept to students is:

A Functional Interface defines what operation is required, while a Lambda Expression provides how that operation should be performed.

For example:

```
@FunctionalInterface
interface Operation {
    int perform(int a, int b);
}
```

defines **what** is required.

Then:

```
Operation add = (a, b) -> a + b;
```

defines **how** it is performed.

Similarly:

```
Operation multiply = (a, b) -> a * b;
```

provides another implementation of the **same functional interface**.

One important correction to your original code

Your original interface says:

```
double operate(int a, int b);
```

but the anonymous implementation says:

```
public void operate(int a, int b)
```

That is a **return-type mismatch** and will produce a compilation error. It should be:

```
public double operate(int a, int b)
```

and then:

```
return Math.pow(a, b);
```

Also, I would use the package name `com.company.lambda` rather than `com.company.lamda`; `lambda` is the standard spelling.

Examples:

```
package com.company.lamda;

public class Demo2 {

    public static void main(String[] args) {

        /*
         * Create an array of String values.
         * The array contains names of different students/persons.
         */
        String[] names = new String[] {
            "Rahul",
            "Rohan",
            "Rajesh",
            "Brajesh",
            "Jayesh",
            "Ajay",
            "Naveen",
            "Parveen",
            "Priya",
            "Riya"
        };

        /*
         * -----
         * Example 1: Display names starting with the letter "R"
         * -----
         *
         * Arrays.stream(names)
         *     -> Converts the String array into a Stream.
         *
         * filter(name -> name.startsWith("R"))
         *     -> Filters the names.
         *     -> Only names beginning with "R" are selected.
         *
         * forEach(System.out::println)
         *     -> Performs an operation on every selected element.
         *     -> System.out::println is a METHOD REFERENCE.
         *     -> It is equivalent to:
         *
         *         name -> System.out.println(name)
         */
        java.util.Arrays.stream(names)
            .filter(name -> name.startsWith("R"))
            .forEach(System.out::println);

        /*
         * Print a separator line to distinguish
         * the output of the two stream operations.
         */
        System.out.println("=====");
    }
}
```

```

/*
 * -----
 * Example 2: Find names containing "iy"
 * -----
 *
 * Arrays.stream(names)
 *     -> Converts the array into a Stream.
 *
 * filter(name -> name.contains("iy"))
 *     -> Selects only those names that contain "iy".
 *
 * forEach(name -> System.out.println(name.toUpperCase()))
 *     -> Performs an operation on every matching name.
 *     -> Converts the name to uppercase before printing it.
 *
 * Here, the Lambda Expression contains the complete action
 * that should be performed for each selected element.
 */
java.util.Arrays.stream(names)
    .filter(name -> name.contains("iy"))
    .forEach(name -> System.out.println(name.toUpperCase()));
}
}

```

Expected Output

```

Rahul
Rohan
Rajesh
Riya
=====
PRIYA
RIYA

```

Concepts Demonstrated

Concept	Code	Purpose
Array	String[] names	Stores multiple names
Stream	Arrays.stream(names)	Creates a stream from the array
Lambda Expression	name -> name.startsWith("R")	Defines the filtering condition
filter()	.filter(...)	Selects elements satisfying a condition
forEach()	.forEach(...)	Performs an operation on each selected element
Method Reference	System.out::println	Short form of a lambda
String Method	startsWith("R")	Checks the beginning of a string
String Method	contains("iy")	Checks whether a string contains a

Concept	Code	Purpose
		sequence
<code>toUpperCase()</code>	<code>name.toUpperCase()</code>	Converts the name to uppercase

Important Point for Students

The following two statements are equivalent:

```
.forEach(name -> System.out.println(name));
```

and

```
.forEach(System.out::println);
```

The second form is called a **Method Reference**.

Similarly:

```
.filter(name -> name.startsWith("R"))
```

means:

Take each `name` from the stream and check whether it starts with "R".

If the result is `true`, the name passes through the `filter()` operation.

The overall flow can be explained as:

```

Array
  ↓
Arrays.stream()
  ↓
Stream of names
  ↓
filter()
  ↓
Matching names
  ↓
forEach()
  ↓
Output

```